```
# This file is automatically generated, edit make Makefile analysis/plots instead
# It's actually a Python header file, but Jupyter/IPyNB understands the "%run" command
# Set the matplotlib backend to inline (for IPyNB)
%matplotlib inline
import sys, os, f, time, glob, h
import numpy as np, scipy.stats
import matplotlib, matplotlib.pyplot as plt, matplotlib.ticker
# Set the default figure size
plt.rcParams['figure.figure_size'] = 10, 7.5
# Force matplotlib to not use any Xwindows backends.
matplotlib.rcParams['backend'] = 'Agg'
# Load the libraries.
import lib.utilsplots, lib.utilsmath, lib.utilsio, lib.utilsastro
# Define a function to print in addition to the{{{
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     64267
           1       0.98      1.00      0.99      4460

    accuracy                           1.00     68727
   macro avg       0.99      1.00      1.00     68727
weighted avg       1.00      1.00      1.00     68727
```

In [30]:
```
# ============================================================
# Confusion Matrix
# ============================================================
import seaborn as sns

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, cmap="Blues", fmt='d',
            xticklabels=["Non-Stress","Stress"],
            yticklabels=["Non-Stress","Stress"])
plt.title("Confusion Matrix")
plt.show()
```

In [31]:
```
# ============================================================
# ROC Curve
# ============================================================
fpr, tpr, _ = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(7,5))
plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}")
plt.plot([0,1],[0,1],'k--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.grid(True)
plt.show()
```

In [32]:
```
# ============================================================
# Training Curves
# ============================================================
plt.figure(figsize=(10,4))
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Training & Validation Accuracy")
plt.legend(["Train","Val"])
plt.show()

plt.figure(figsize=(10,4))
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Training & Validation Loss")
plt.legend(["Train","Val"])
plt.show()
```

In [33]:
```
# ============================================================
# Attention Visualization
# ============================================================
sample = X_test[:1]
att = viz.predict(sample).flatten()

plt.figure(figsize=(12,4))
plt.plot(att)
plt.title("Attention Weights Over Time")
plt.xlabel("Time Step")
plt.ylabel("Importance")
plt.show()

plt.figure(figsize=(10,2))
plt.imshow(att.reshape(1,-1), cmap="viridis", aspect="auto")
plt.colorbar()
plt.title("Attention Heatmap")
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 416ms/step
```

In [34]:
```
model.save("proactive_stress_model.h5")
print("Saved proactive_stress_model.h5")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_mode
l(model)`. This file format is considered legacy. We recommend using instead the native Keras forma
t, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
Saved proactive_stress_model.h5
```